WEB AUTOMATION GRADUATION PROJECT

Project Plan & Implementation Roadmap

Target website	Your Store — awesomeqa.com
Automation framework	Selenium WebDriver + Java + TestNG (recommended)
Reporting	ExtentReports HTML execution report
Required scope	Automate the specified cases, add 20 original cases, and generate a report.

Prepared as a practical step-by-step plan. Update the team names, tool choices, and final test count to match your instructor’s requirements.

1. Understand the Requirements

The assignment has three deliverables: automate the listed baseline scenarios, create and automate 20 additional test cases, and produce an ExtentReports report for the test execution. The image lists 12 baseline scenarios, while the requirement says “10 test cases.” Confirm with the instructor whether all 12 listed scenarios are expected. This plan assumes all 12 will be automated unless clarified.

Baseline scenarios shown in the assignment

#	Scenario	Coverage to include
1	Registration	Valid registration; invalid/missing fields; duplicate email if supported
2	Login	Valid login; invalid password/email; empty fields
3	Forgot password	Submit registered/unregistered email; validation and confirmation message
4	Search product	Search an existing product and a product with no results
5	Switch currency	Switch USD ↔ EUR and verify displayed prices/currency symbol
6	Select main/subcategories	Open randomly selected main and subcategories
7	Hover main/subcategories	Hover menus and verify submenu visibility
8	Select different categories	Navigate across multiple categories and verify category page
9	Wishlist	Add an item to wishlist and verify it appears
10	Compare list	Add items to compare and verify comparison list
11	Shopping cart	Add item(s), verify cart contents and quantities
12	Successful order/payment	Complete checkout only with an authorized test payment method/environment

2. Recommended Tools and Project Structure

Recommended stack: Java, Maven, Selenium WebDriver, TestNG, WebDriverManager (or a compatible browser driver setup), and ExtentReports. If your course has standardized on Python/pytest or another stack, use the course stack instead; the phases below still apply.

Suggested structure

web-automation-project/
■■■ pom.xml
■■■ src/test/java/
■ ■■■ base/ (browser setup and teardown)
■ ■■■ pages/ (Page Object classes)
■ ■■■ tests/ (test cases grouped by feature)
■ ■■■ utilities/ (waits, screenshots, test data)
■ ■■■ listeners/ (TestNG + ExtentReports integration)
■■■ src/test/resources/ (configuration and test data)
■■■ reports/ (generated HTML report; normally ignored by Git)

Use Page Object Model (POM) to keep locators and page actions separate from test assertions. Prefer explicit waits over fixed sleep calls. Keep passwords and payment details out of source code.

3. Execution Roadmap

Phase	Tasks	Exit criteria
1 — Reconnaissance	Open the target website manually; map registration, login, search, categories, currency, wishlist, compare, cart and checkout. Record stable selectors and expected behavior.	All target flows documented; no automation assumptions left unchecked.

Phase	Tasks	Exit criteria
2 — Setup	Create Maven project; add Selenium, TestNG and ExtentReports dependencies; configure browser; verify one smoke test can open and close the site.	A clean test run works on the team's machine.
3 — Framework	Create BaseTest, page objects, explicit-wait helpers, configuration, test data and screenshot-on-failure support.	Framework can run tests independently and clean up the browser reliably.
4 — Baseline cases	Implement the 12 scenarios listed above (or the 10 confirmed by the instructor). Split positive and negative cases into separate tests where appropriate.	Baseline tests are repeatable and have clear assertions.
5 — Add 20 cases	Design, review, automate and label 20 additional cases; avoid simply duplicating the baseline cases.	20 additional cases are documented and automated.
6 — Reporting	Attach ExtentReports to test lifecycle; record pass/fail/skip, duration, exceptions and screenshots for failures.	A readable HTML report is generated for each run.
7 — Stabilization	Run the suite repeatedly; fix flaky selectors/waits; confirm test data cleanup; check for accidental dependency between tests.	Consistent results across at least two full runs.
8 — Submission	Prepare README, test-case matrix, source code, dependencies, report and a short demo/presentation.	Another person can run the project using the README.

4. Proposed 20 Additional Test Cases

These are candidate additions. During reconnaissance, confirm that each feature exists on the current site; replace unsupported cases with equivalent valid cases rather than forcing tests against absent functionality.

Feature	#	Additional test case
Registration	1	Password below minimum length is rejected.
Registration	2	Password and confirmation mismatch shows validation.
Registration	3	Malformed email address is rejected.
Login	4	Email with leading/trailing spaces is handled correctly.
Login	5	Password field masks typed characters.
Forgot password	6	Empty email submission displays validation.
Search	7	Search ignores letter case or behaves consistently.
Search	8	Search query with no matches shows a clear empty state.
Currency	9	Selected currency remains consistent when navigating to a product page.
Categories	10	Browser Back returns to the previous category page.
Categories	11	Category page displays a title consistent with the selected category.
Product details	12	Open a product and verify name, price and key details are visible.
Product details	13	Product image/gallery loads and has no broken image indicator.
Wishlist	14	Removing a product from wishlist removes it from the list.
Compare	15	Removing an item from compare list updates the list.
Cart	16	Updating quantity recalculates subtotal correctly.
Cart	17	Removing a cart item updates cart contents and total.
Cart	18	Empty cart displays an appropriate empty-state message.
Checkout	19	Required checkout fields show validation when left blank.
Checkout	20	Order summary totals are consistent with cart items and shipping/tax rules.

5. Test Case Documentation Template

For every test, document the following before or while automating it:

Field	What to record
Test Case ID / Title	Unique ID and concise scenario name
Preconditions	Required account state, selected currency, cart state, etc.
Test Data	Non-sensitive test inputs; use dedicated test accounts
Steps	Numbered actions a tester or script performs
Expected Result	Observable, specific behavior that should occur
Actual Result / Status	Observed behavior; Pass, Fail, or Blocked
Evidence	Screenshot, exception, or report entry when useful

6. ExtentReports: Reporting Checklist

• Create the report at the start of the test run and flush/write it at the end.

• Create a report entry for each test with its name and feature group.

• Log the test steps or important checkpoints and expected/actual outcomes.

• On failure, capture the exception and a screenshot where possible.

• Include execution date/time, total tests, passed, failed and skipped counts.

• Open the generated HTML report in a browser and verify links, screenshots and status colors.

• Keep generated reports out of source control if they contain test data or environment details; submit a clean sample report as required.

7. Quality, Safety and Reliability

• Use only accounts and test data you are authorized to use.

• Do not run repeated checkout/payment tests against real payment methods. Use a sandbox/test payment option and confirm permission first.

• Avoid storing real passwords, personal information, or payment data in code or reports.

• Use stable IDs/data attributes where available; avoid brittle absolute XPath selectors.

• Use explicit waits for elements and page state; do not rely on arbitrary sleep durations.

• Make tests independent where possible and reset cart/wishlist/test data between runs.

• If the public demo site changes or is unavailable, document the issue and ask the instructor whether a local/demo environment is acceptable.

8. Suggested 2-Week Schedule

Day	Focus	Deliverable
1	Review requirements and explore website	Feature map + clarified scope
2	Create project and verify browser setup	One passing smoke test
3–4	Build base framework and page objects	Reusable framework
5–7	Automate baseline scenarios	Baseline suite
8–10	Design and automate 20 new cases	20-case additional suite
11	Integrate ExtentReports	HTML report with test results
12	Stabilize and rerun suite	Consistent run + defect notes
13	README and test documentation	Reproducible setup instructions
14	Final run and presentation prep	Submission package + demo

9. Final Submission Checklist

■ Source code compiles and runs from a clean checkout.

■ Baseline cases are automated (confirm whether 10 or all 12 are required).

■ Exactly 20 additional cases are documented and automated.

■ Assertions check outcomes, not only that a click succeeded.

■ ExtentReports HTML file is generated and opens correctly.

■ Screenshots and error logs help explain failures.

■ README explains Java/browser requirements, setup, command to run tests, and report location.

■ No secrets or real payment details are included.

■ Final demonstration shows at least one positive case, one negative case, and the HTML report.

Important note: the brief says “10 Test cases” but visibly lists 12 numbered scenarios. Confirm this discrepancy with the instructor before finalizing the test suite.